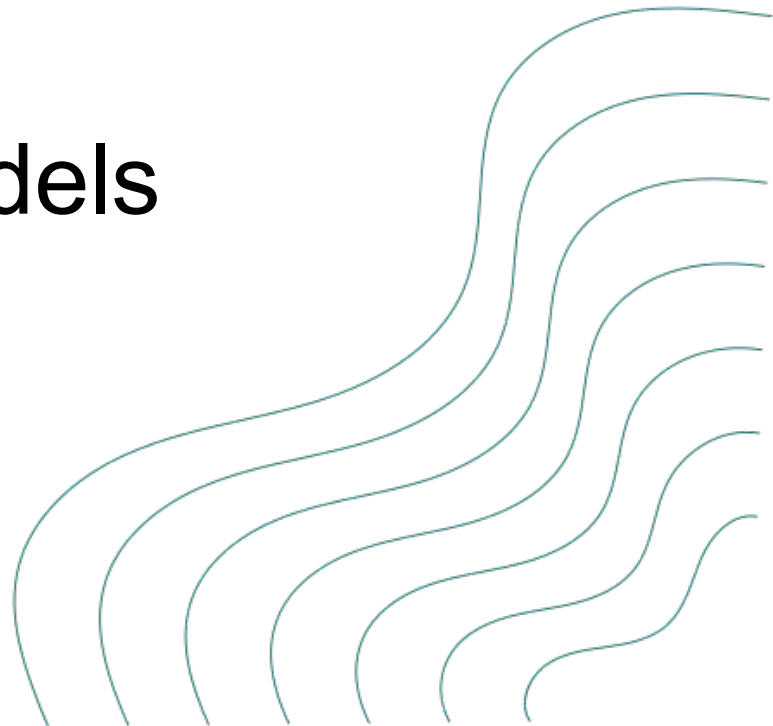




Introduction to Deep Learning Models and Applications

Golnaz Mesbahi
MSc, Computer Science



Outline Part 1: Fundamentals of Neural Networks

Training and Testing

Processes to develop and evaluate the model's performance.

Perceptron

The basic unit of a neural network that mimics a biological neuron.

Activation Functions

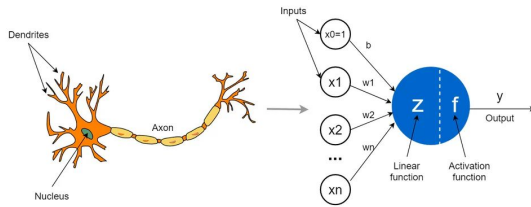
Functions that determine the output of a perceptron

Weights and Gradient Descent

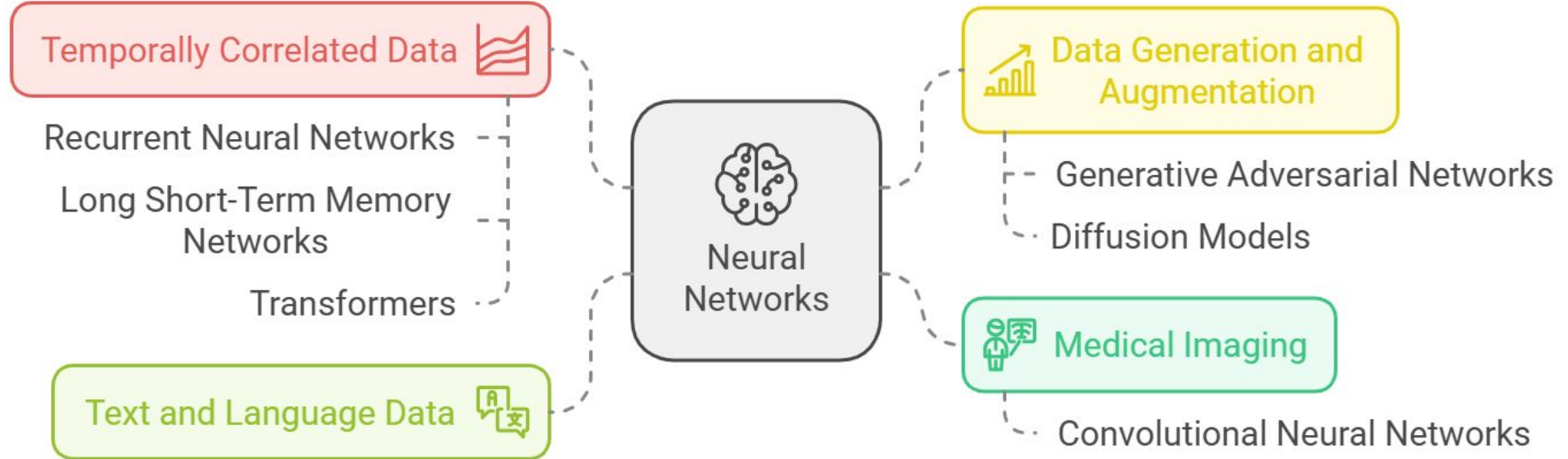
Parameters adjusted during training to minimize error.

Hyperparameters

Configuration settings that govern the training process.



Outline Part 2: Applications of Neural Networks



From previous sessions...

What do we aim to learn?

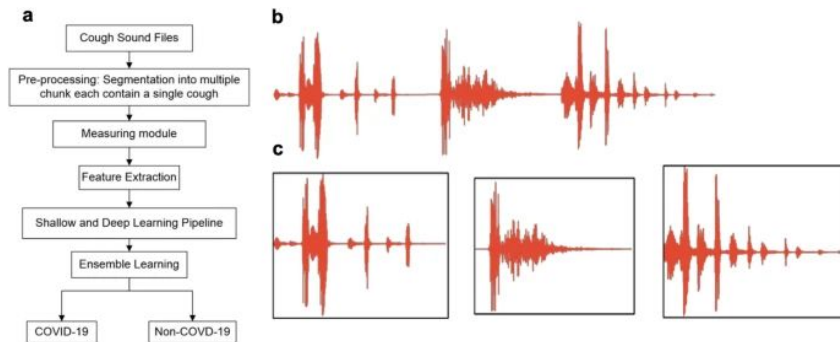
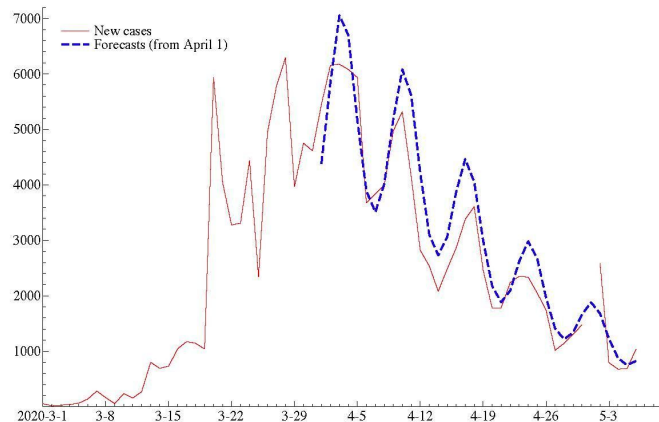
X (input, features) $\rightarrow$ [model] $\rightarrow$ **Y** (output, label)

- **Regression Problems**

- predict continuous value **Y**
- Predict number of new covid cases in the next month

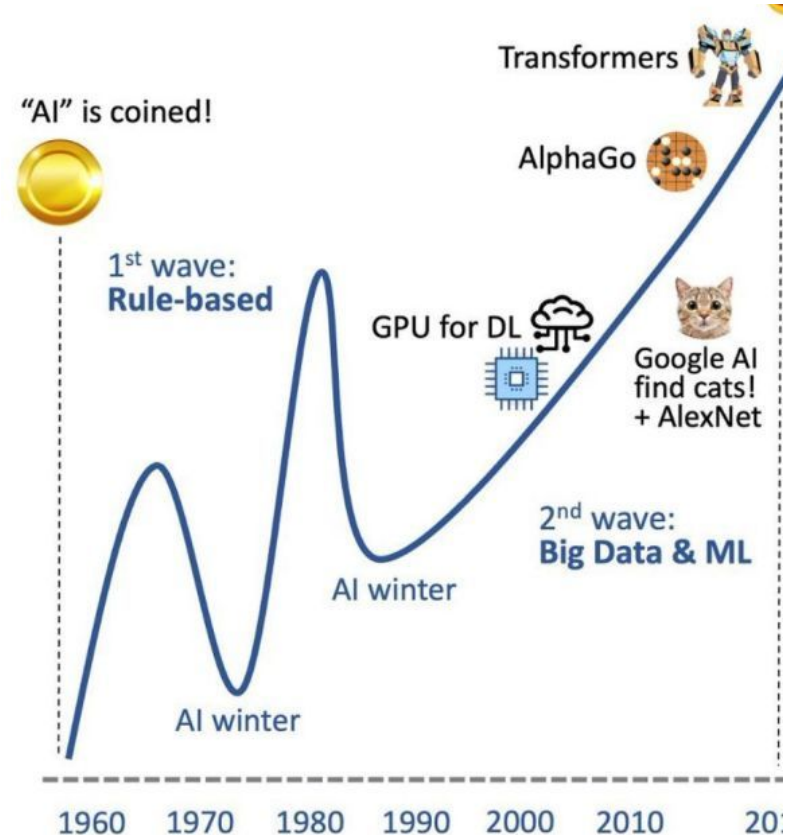
- **Classification Problems**

- Predict a discrete class label **Y**
- Does the patient have covid based on the sound of his coughing?
Class: covid, non-covid



How did we move from previous solutions to NNs?

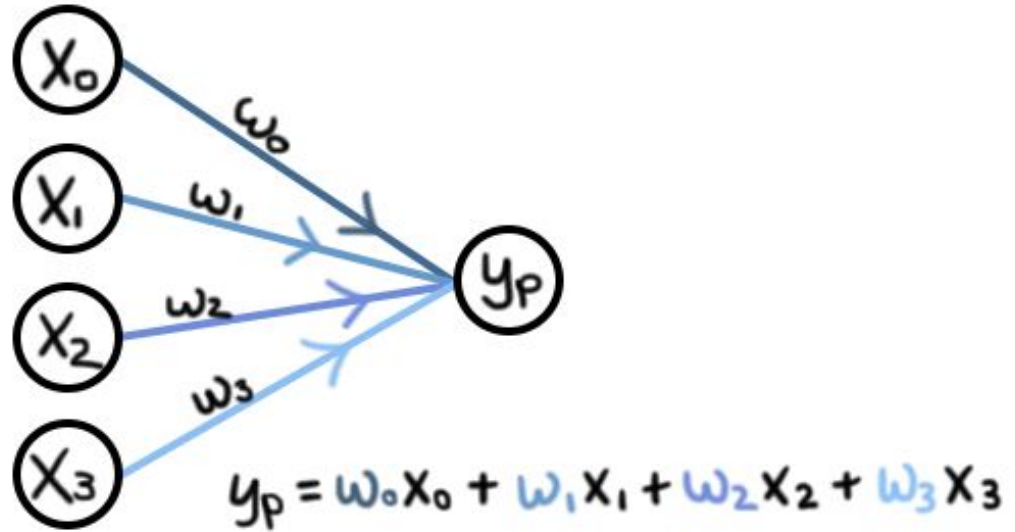
- Complex patterns and **nonlinear relationships**
- Large datasets and **high-dimensional data** (the need for data driven solutions)
- **Unstructured data** (image, text)
- Programming and **hardware advances**



**Let's visit some models that
help us solve these
problems!**

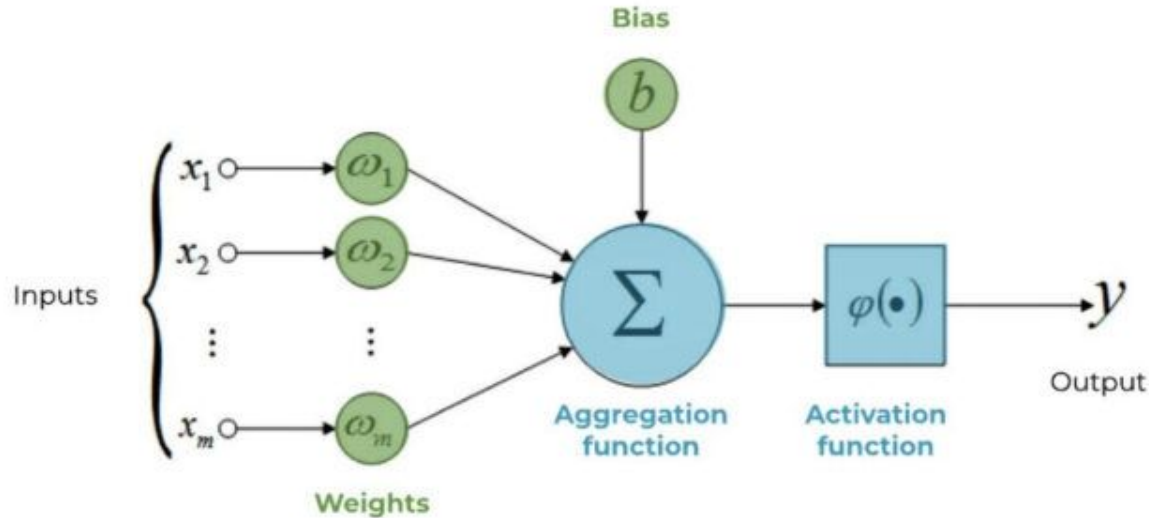
Neural Networks: **Perceptron**

- Linear regression or Logistic Regression can be thought of as single layer Neural Network
- The smallest unit of computation in neural networks



$$y = \text{activation}(y_p)$$

Neural Networks: Activation Functions

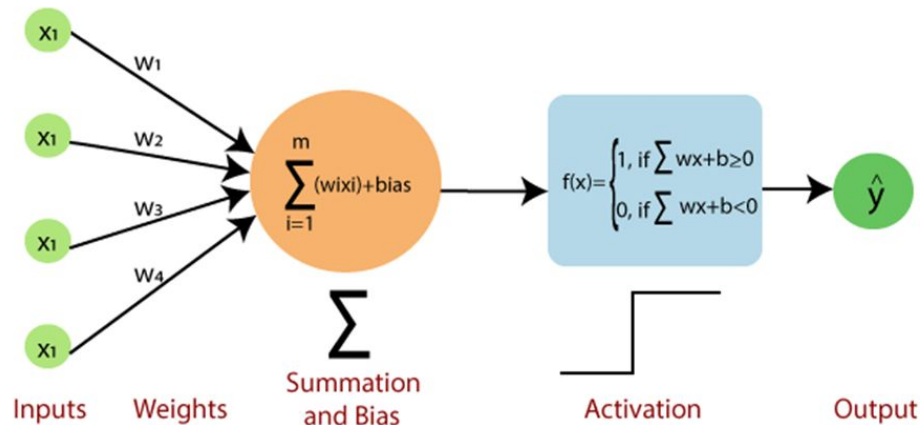


Learnable through time
Fixed by the AI expert

$$y = \varphi(b + \sum x_i \omega_i)$$

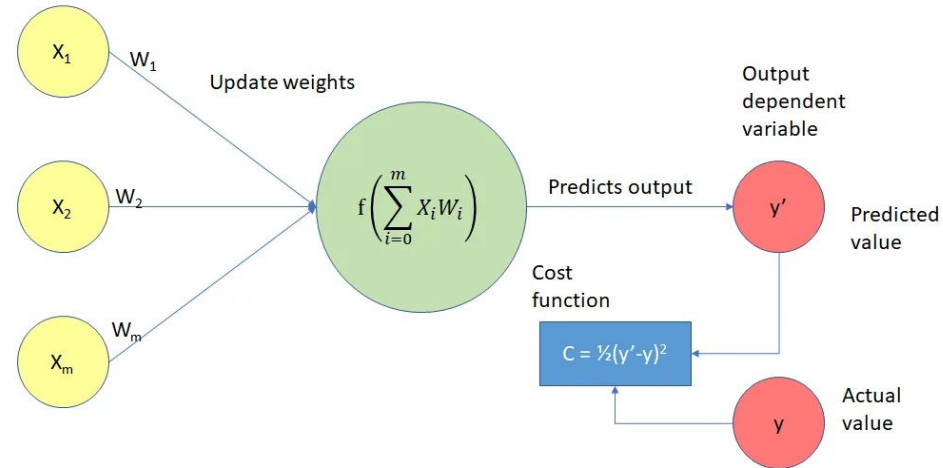
Neural Networks: **Weights**

- a system of knobs that we can manipulate to **optimize our model**
- Indicates how important a feature input is
- Initially, the weights are **set randomly** (and by randomly, we really don't mean that!)
- In order to learn, **the weights must be adjusted to minimize the cost function**



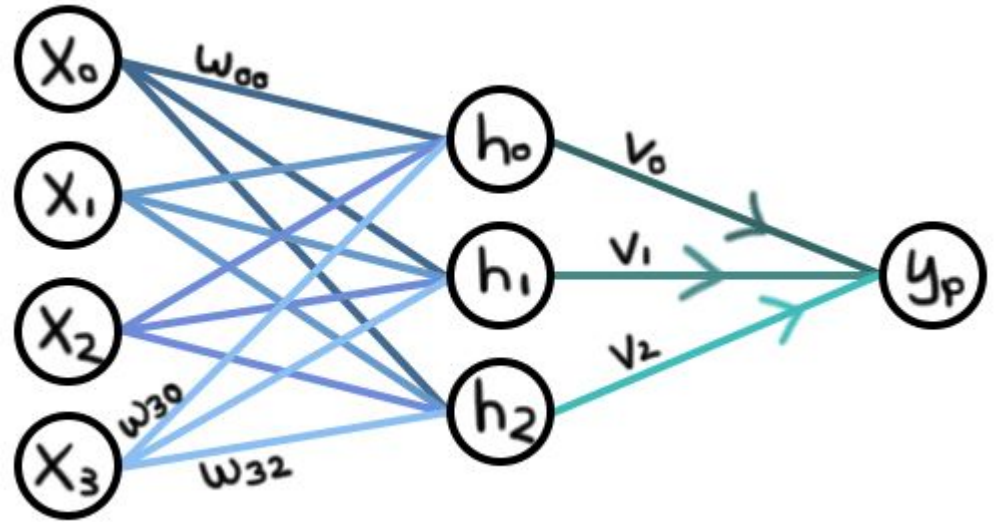
Neural Networks: Cost Function

- Represents the **error between predicted and expected values**
- The goal is to **minimize a cost or loss function**
- Depending on the problem, there are different loss functions to use:
 - **mean squared error** (regression)
 - **Cross entropy loss** (classification)



Moving from perceptron to neural network

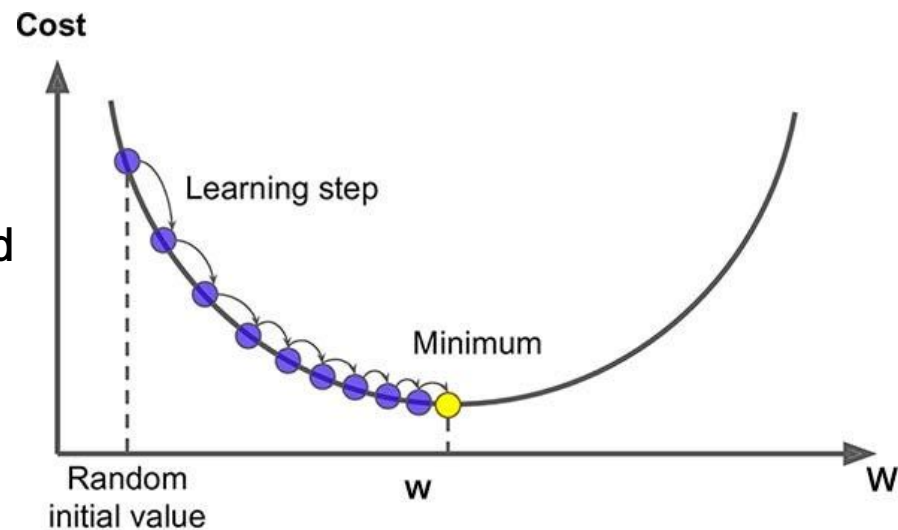
- Adding layers of neurons
- The first layer is the input layer, it receives the features of your input
- The last layer is the output layer. It gives you the desired output



How to Update the weights: Gradient Descent

A method of **updating the weights** to **minimize loss**

1. **Random initialization of weights** = Choose a random starting point
2. **Forward pass** = calculate the outputs with the current weights
3. **Calculate loss** = how much the predicted output is different from the desired output



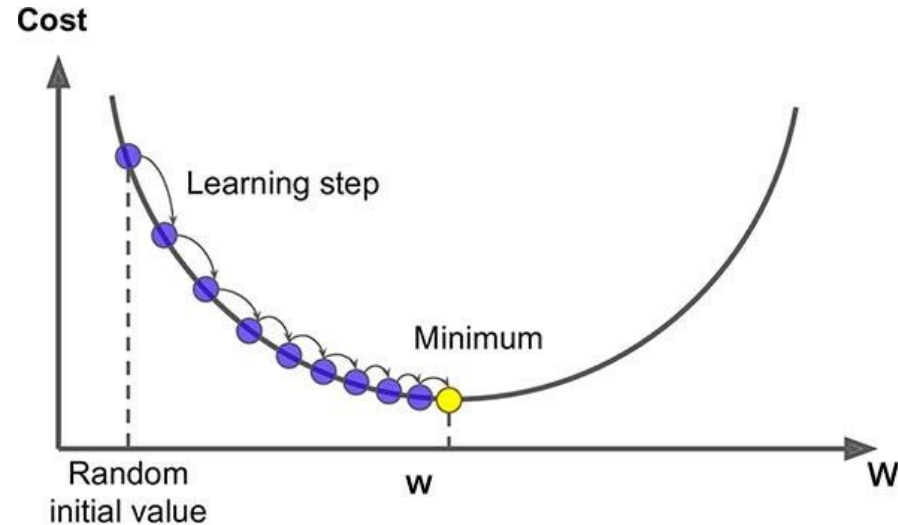
How to Update the weights: Gradient Descent

4. Calculate the **gradients** = slope, **measures the change in all weights with regard to the change in error.**

5. Multiply by **learning rate** = step size

6. **Backpropagate** = update weights

$$W := W + \alpha \nabla L$$



How to Update the weights: Gradient Descent

$$W := W + \alpha \nabla L$$

New weights = **old weights** + **learning rate** * **gradients**

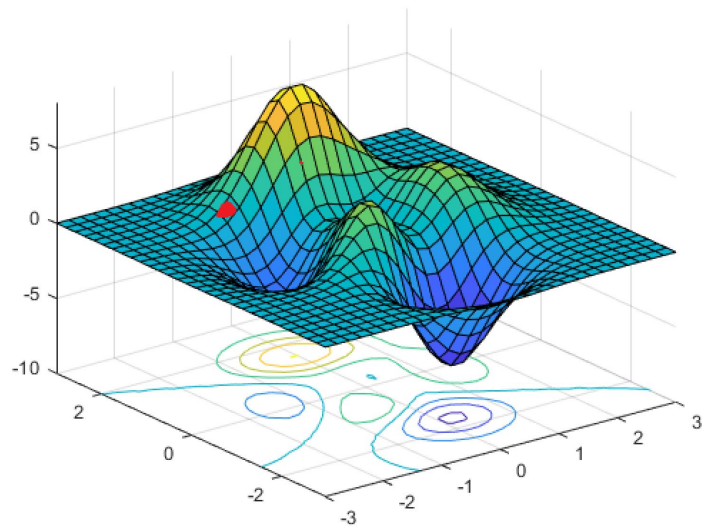
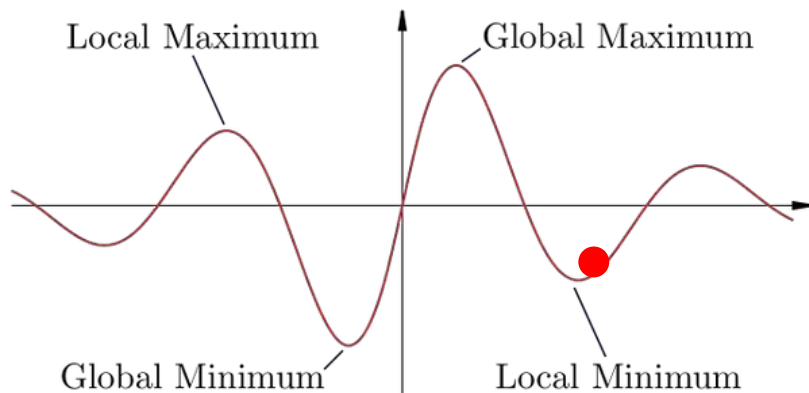
New weights = **old weights** + **how fast to change the weights** * **the direction to change the weights to minimize the loss**

As loss gets smaller, gradients get smaller and therefore the weights change less

Neural Networks: Gradient Descent

So, are we done? No, it is rarely that simple

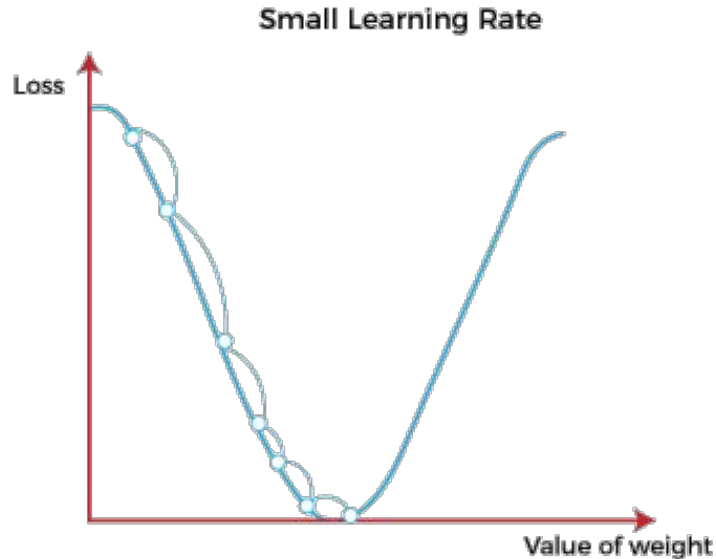
- Can get stuck in **local minimum**.
- Typically involves **multiple dimensions**



Gradient Descent: Learning Rate

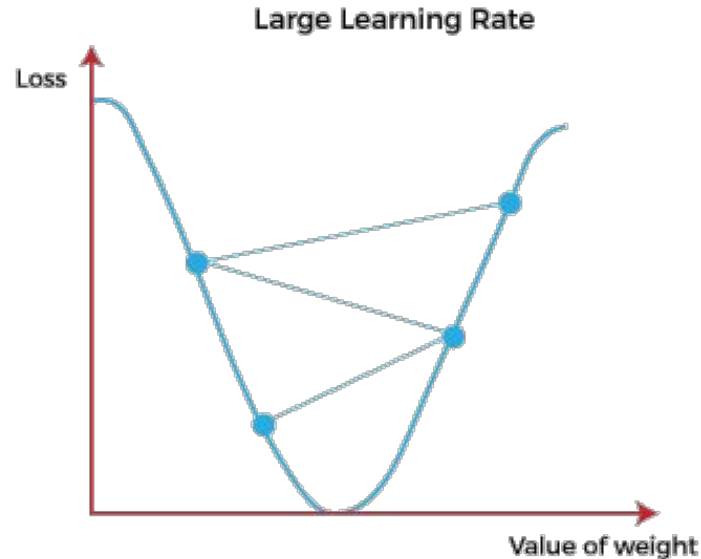
If learning rate is **too small**

It will take too long to find the minimum



If learning rate is **too big**

It will skip over the minimum entirely (We say the algorithm diverged)



Learning Rate: How to solve the problem of too big or too small steps?

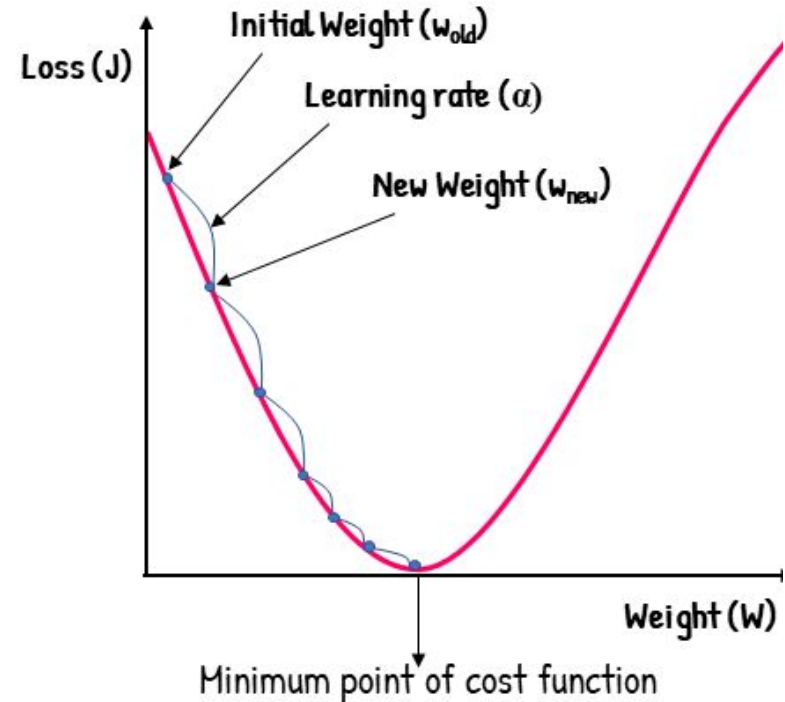
Learning rate decay

Decreases step size as loss decreases

Variety of decay functions

- Exponential
- Momentum
- AdaGrad
- **Adam**

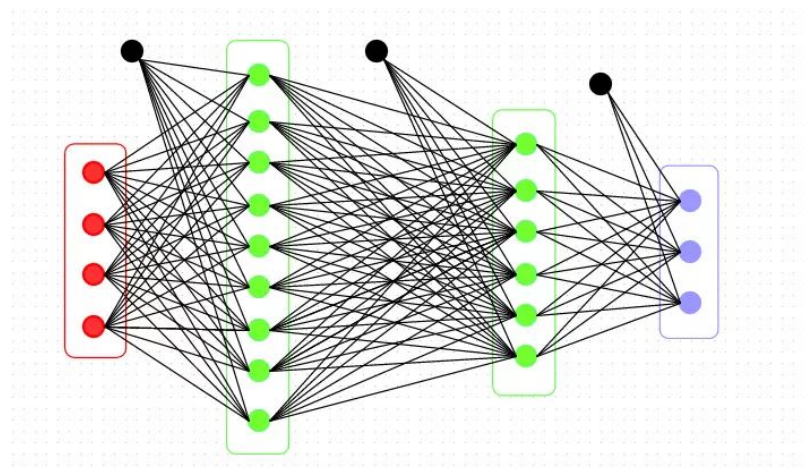
Saves time and finds the minimum!



Neural Networks: Hyperparameters

Fine tune the training process by adjusting hyperparameters:

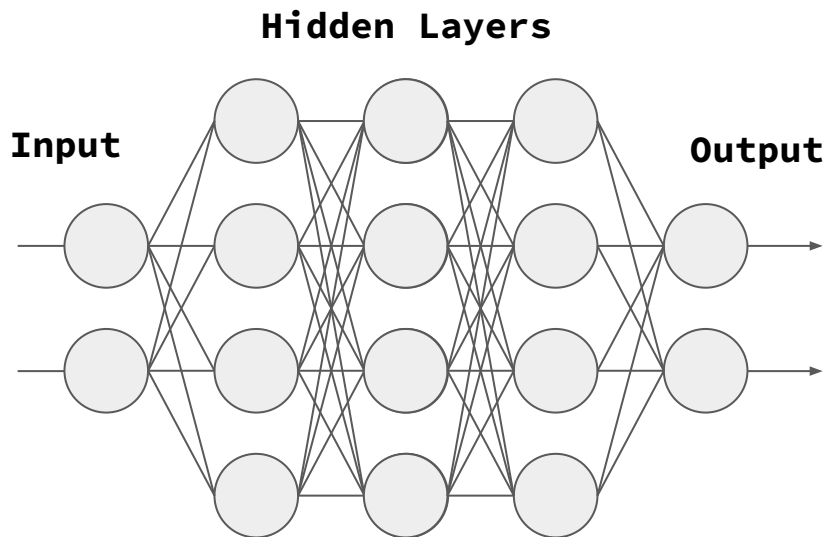
- Learning rate
- Loss functions
- Number of training sessions (epochs)
- Number of layers (how deep should the network be?)
- Number of perceptrons in every layer



Neural Networks: Putting it all together

Training an AI model

1. Random initialization of weights
2. Training iteration
 - a. Input data
 - b. Propagate data through the network via activation functions
 - c. Output values (the prediction)
 - d. Calculate loss (the accuracy)
 - e. Backpropagate to update weights
3. Repeat for x number of iterations until
loss is at a minimum
 - a. If unsuccessful, adjust hyperparameters and retry
4. At this point your weights are optimized
and your model is trained and ready to use!



Neural Networks: An Example, Heart Failure Prediction

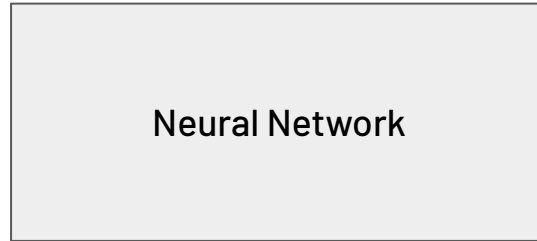
Step by Step Artificial Neural Network (NN)

Heart Failure Prediction

ejection_fraction	high_blood_pressure	platelets	serum_creatinine	serum_sodium	sex	smoking	time	DEATH_EVENT
20	1	265000.00	1.9	130	1	0	4	1
38	0	263358.03	1.1	136	1	0	6	1
20	0	162000.00	1.3	129	1	1	7	1
20	0	210000.00	1.9	137	1	0	7	1
20	0	327000.00	2.7	116	0	0	8	1
40	1	204000.00	2.1	132	1	1	8	1
15	0	127000.00	1.2	137	1	0	10	1
60	0	454000.00	1.1	131	1	1	10	1
65	0	263358.03	1.5	138	0	0	10	1
35	1	388000.00	9.4	133	1	1	10	1

Neural Networks: An Example, Heart Failure Prediction

ejection_fraction	high_blood_pressure	platelets	serum_creatinine	serum_sodium	sex	smoking	time
20	1	265000.00	1.9	130	1	0	4



Heart failure: true

**But... One step is missing before deploying
your neural network.**

Neural Networks: Evaluation

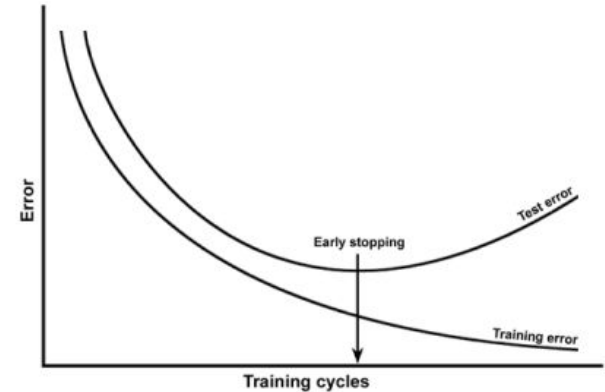
1 - End the training (fix the weights, **no more updates** to the parameters)

2 - choose a **metric** to report how good your model works

3 - prepare a new dataset apart from what your model was trained on (this prevents **overfitting**)

4 - Run the model on the test dataset and compute the chosen metric to assess its performance

Metrics: M.S.E , AUC, average performance, average accuracy



Let's brainstorm! Evaluation Metrics

What are some evaluation metrics for medical applications of neural networks?

Confusion Matrix

	Predicted 	Predicted 
Actual 	4 [TN]	2 [FP] 
Actual 	1 [FN] 	3 [TP] 

Precision

Recall / True Positive Rate / Sensitivity

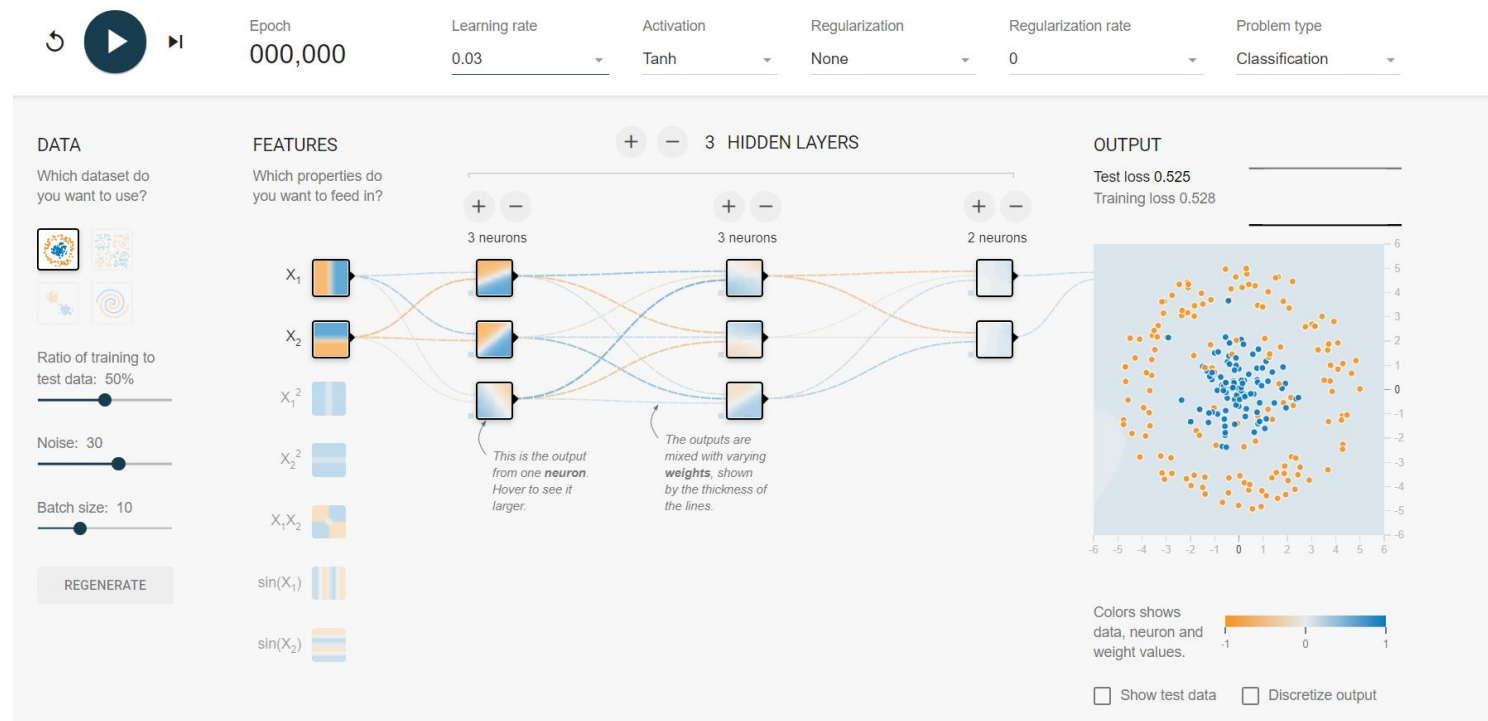
$$\text{Precision} = \frac{3}{3+2} = 0.6 \Rightarrow 60\%$$

$$\text{Recall} = \frac{3}{3+1} = 0.75 \Rightarrow 75\%$$

$$F1 \text{ Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 * 0.6 * 0.75}{0.6 + 0.75} = 0.6666 = 0.67$$

Let's visit a cool playground:

[A Neural Network Playground](#)



Some more exercises

<https://covizdemo.vercel.app/>

What is the process of training a Neural Network?

1 - **Setting up**: choose the details of your NN architecture based on your **problem**

- activation functions
- preprocessing
- weight initialization

2. **Training dynamics**: update your model in the **learning process**

- Weight updates
- hyperparameter tuning

3. **Evaluation**: Does your model match your expectations? Can it generalize well to **unseen data**?

- Overfitting, underfitting, transfer,

Neural Networks for Medical Imaging

Let's brainstorm!

What are some examples of the imaging data in medicine?

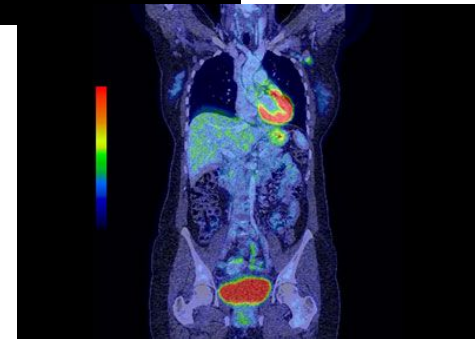
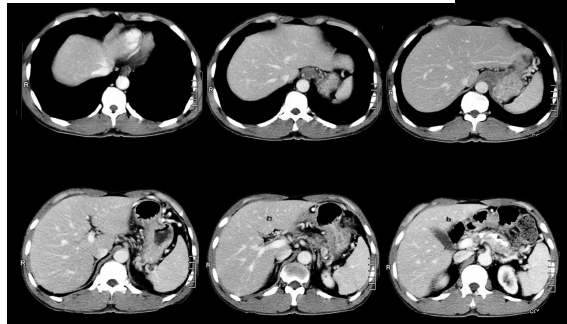
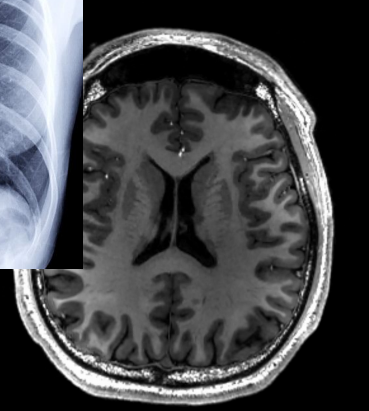
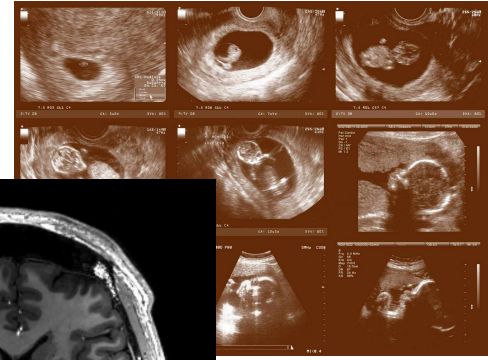
Some examples

Types of data

- Ultrasound
- MRI
- X-ray
- CT
- PET

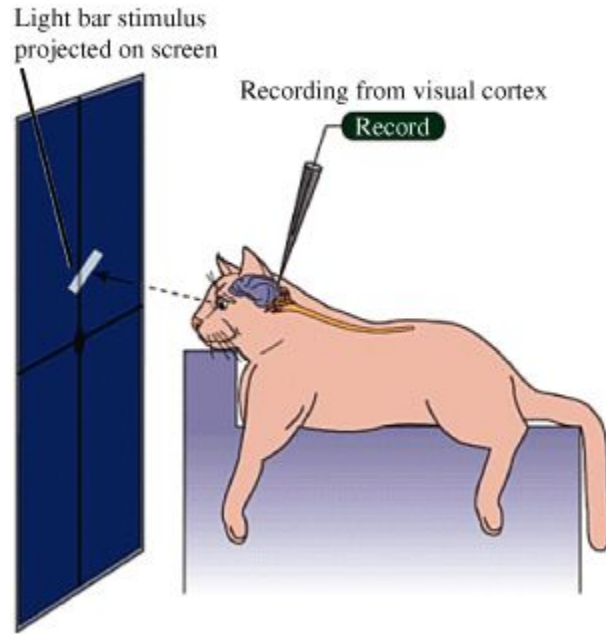
Applications in medicine

- Radiology
- Pathology
- Dermatology



Convolutional Neural Networks

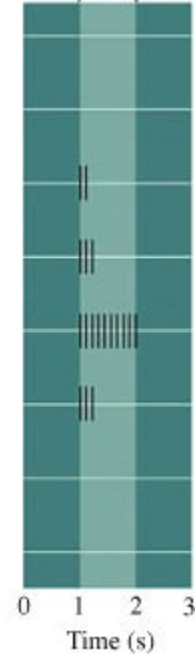
A Experimental setup



B Stimulus orientation



Stimulus presented



How Brain Processes Visual Information

- **Hierarchical Feature Extraction**

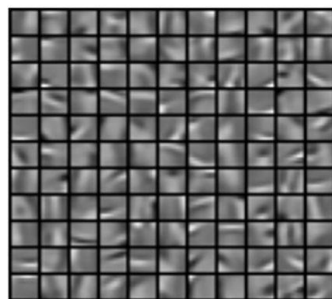
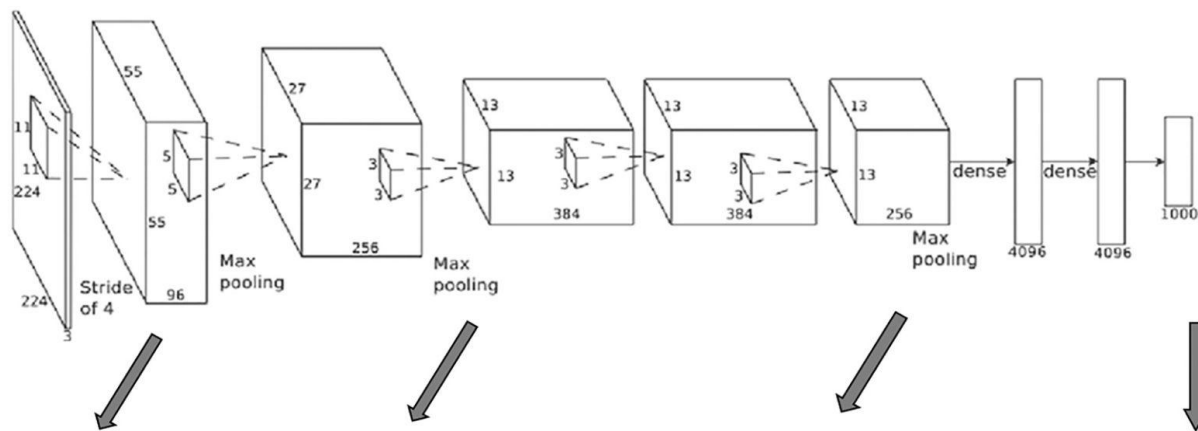
- neurons in lower layers detecting basic features (e.g., edges) and neurons in higher layers combining these features to recognize complex shapes.

- **Translation Invariance**

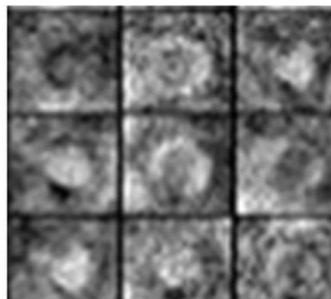
- Ability to detect features regardless of location

From Hubel and Wiesel's Experiments to CNNs

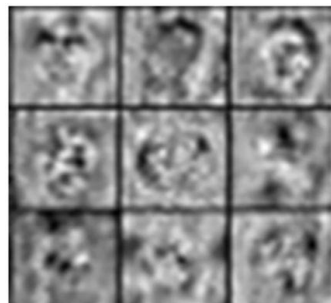
- In the brain, there are **specialized neurons or groups of neurons** that are tuned to detect specific types of features in the visual input: edge detectors, shape detectors
- In CNNs, there are **small, trainable matrices** that scan across an image to detect patterns like edges, textures, or colors.
- A filter is learned in a CNN



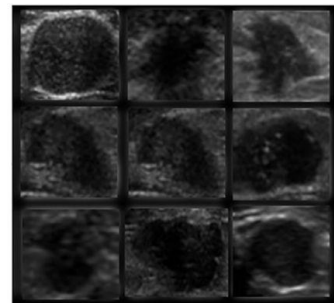
Low-level feature



Mid-level feature



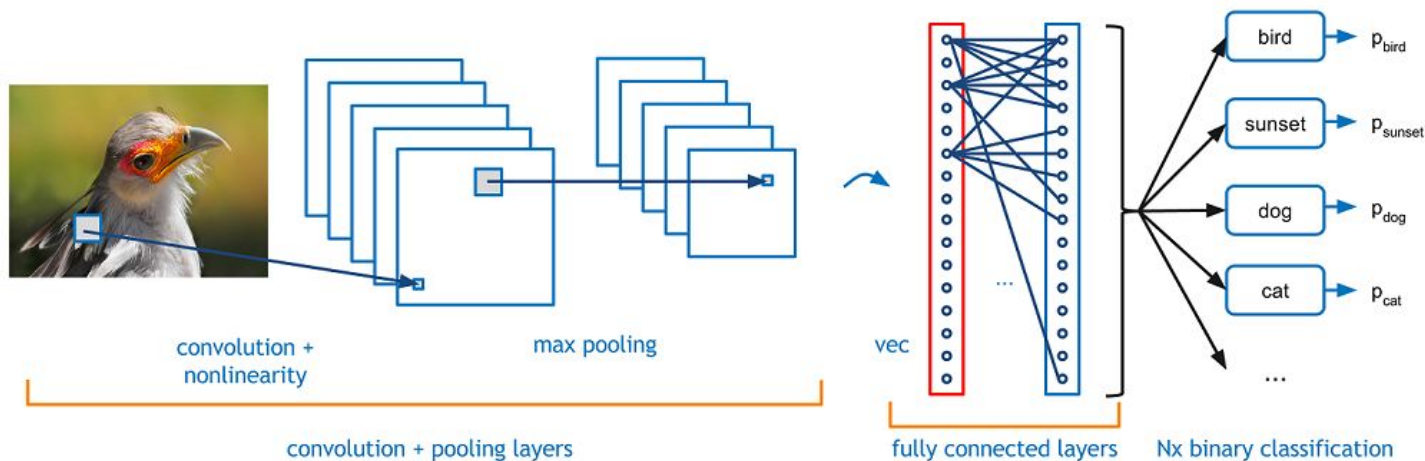
High-level feature



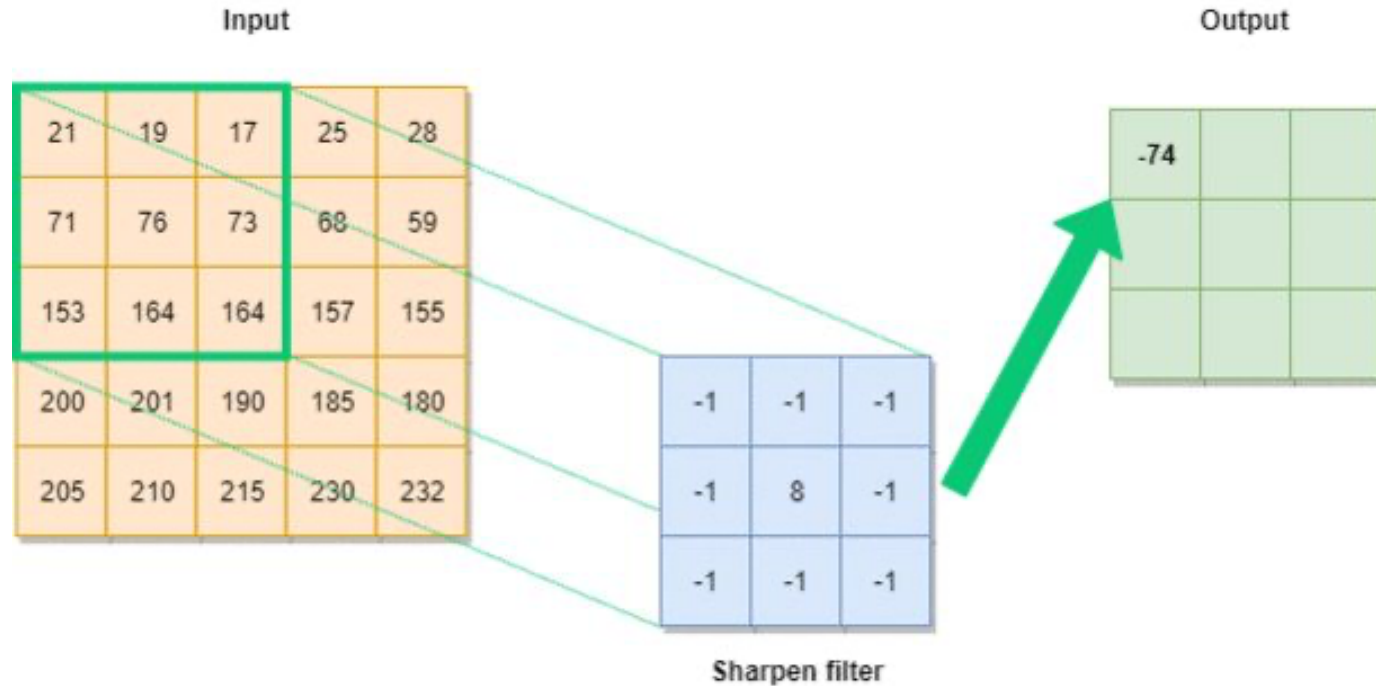
Fc: Object classes

Convolutional Neural Networks

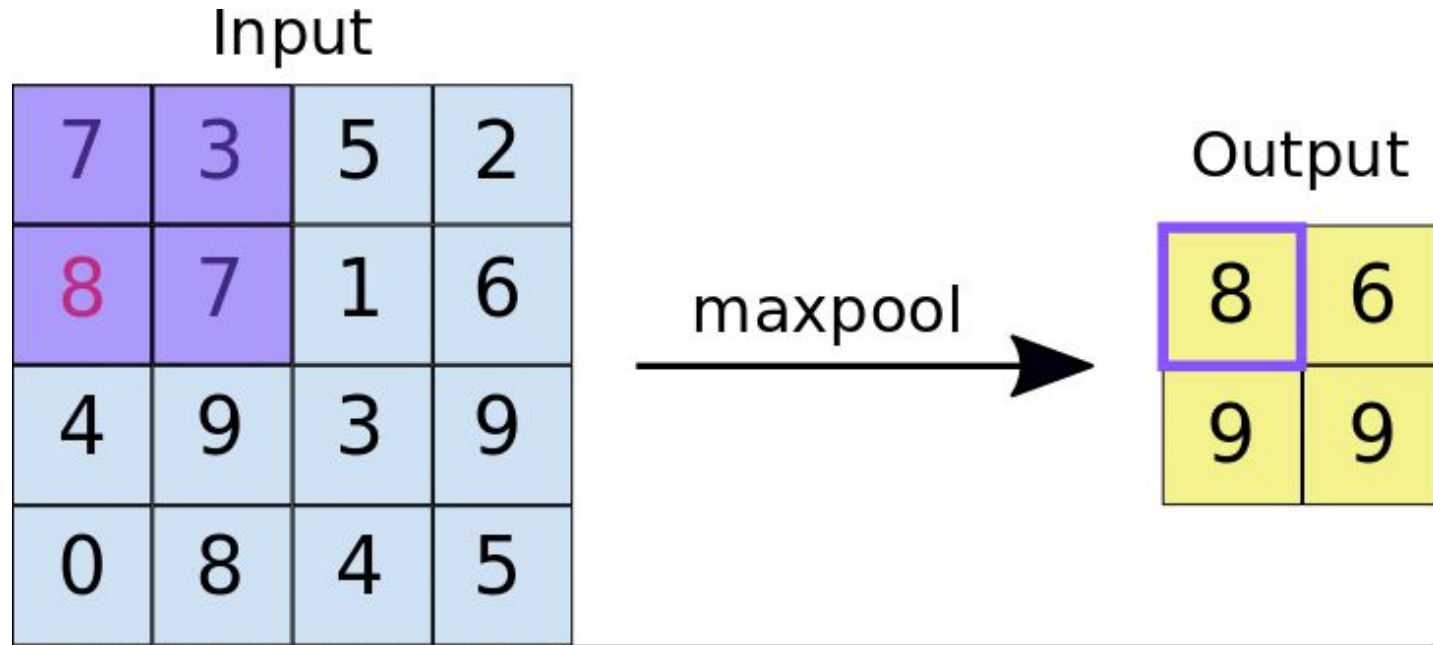
- Hubel and Wiesel inspired the idea of CNNs.
- The same **filter** behavior happens in multiple **receptive fields**
- CNNs are suitable for data that has **spatio temporal dependencies**
- Some design choices (hyperparameters): Number of filters, padding, number of strides, etc.



CNNs: Convolution Layer



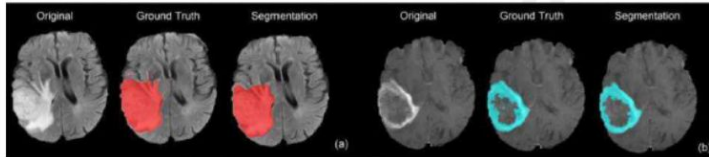
CNNs: Pooling Layer



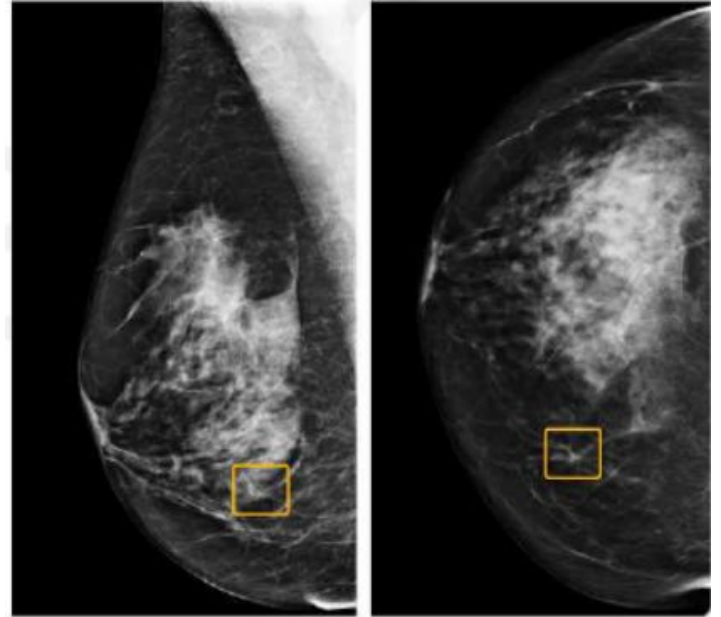
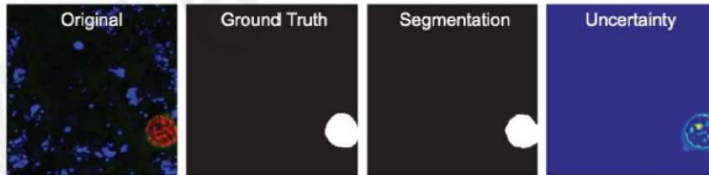
CNN applications

- Classification
- Image Segmentation

Brain Tumors
Dong+ *MIUA* 2017.



Malaria Infection
Soleimany+ *arXiv* 2019.



Breast cancer case missed by
radiologist but detected by AI

Let's go to the playground

<https://poloclub.github.io/cnn-explainer/>

Neural Networks for Temporally Correlated Data

Let's brainstorm!

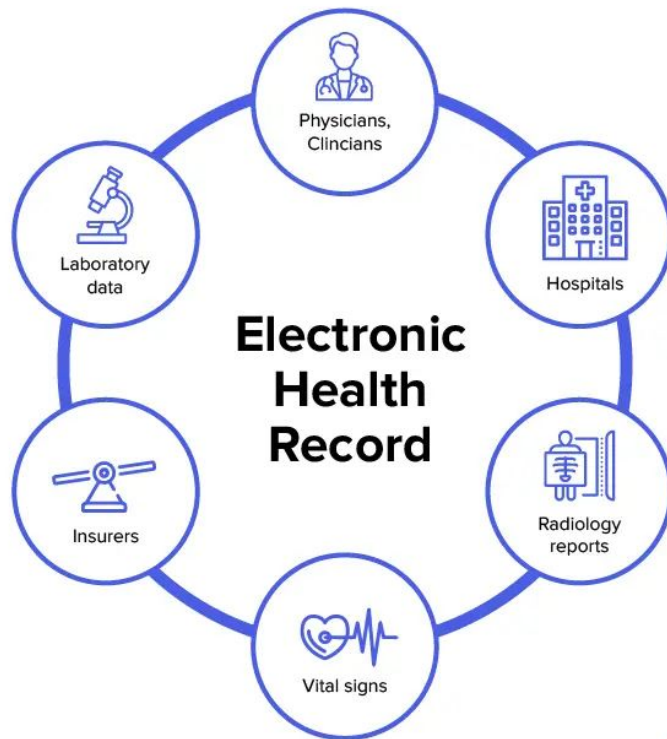
What are some examples of temporally correlated data in medicine?

Temporally Correlated Data

- Electronic Health Records (EHRs)
- electroencephalogram (EEG)
- Respiratory rate monitoring
- Physical activity monitoring
- Medication dosage

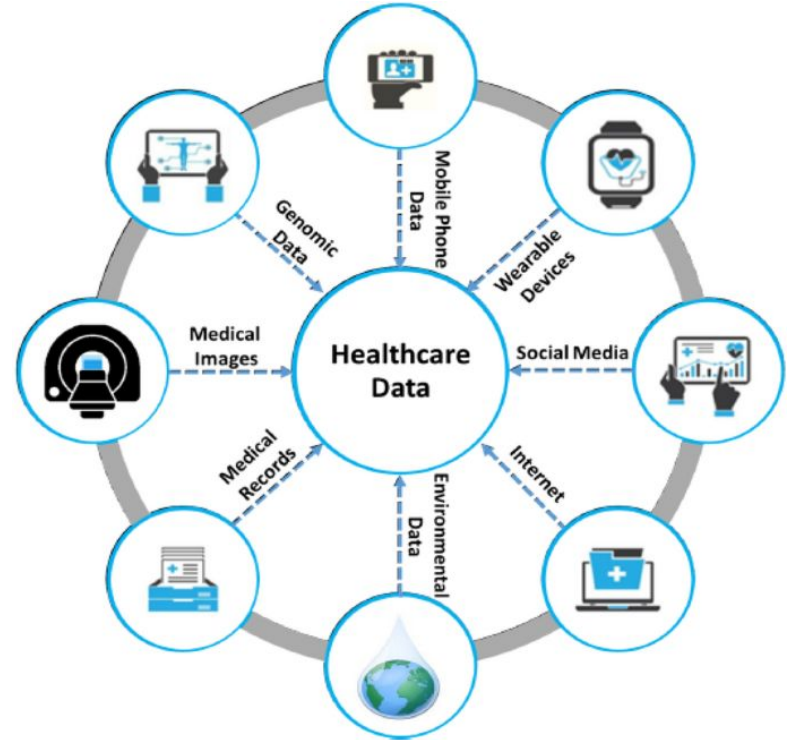
Applications:

Prediction and diagnosis tasks



Neural Networks for temporally correlated

- Recurrent Neural Networks (RNN)
- Long Short Term Memory (LSTM)
- Gated Recurrent Unit (GRU)
- Attention Units
- Transformers



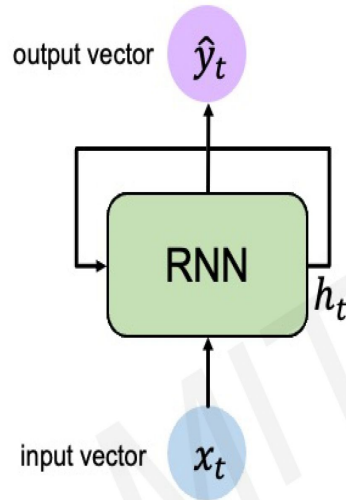
Let's brainstorm!

What are some challenges of time series data?

what are some challenges of time series data?

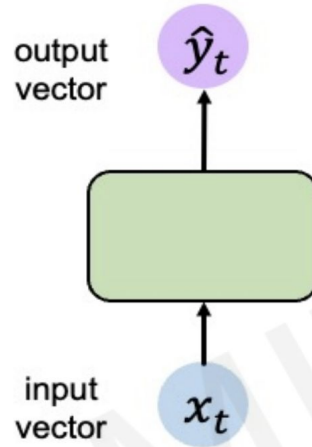
- Irregular time intervals (e.g irregular visits of the patient)
- Missing data
- High dimensionality and large volume
- non-stationarity

1 - Recurrent Neural Networks (RNNs)



$$\hat{y}_t = f(x_t, h_{t-1})$$

output input past memory



$$\hat{y}_t = f(x_t)$$

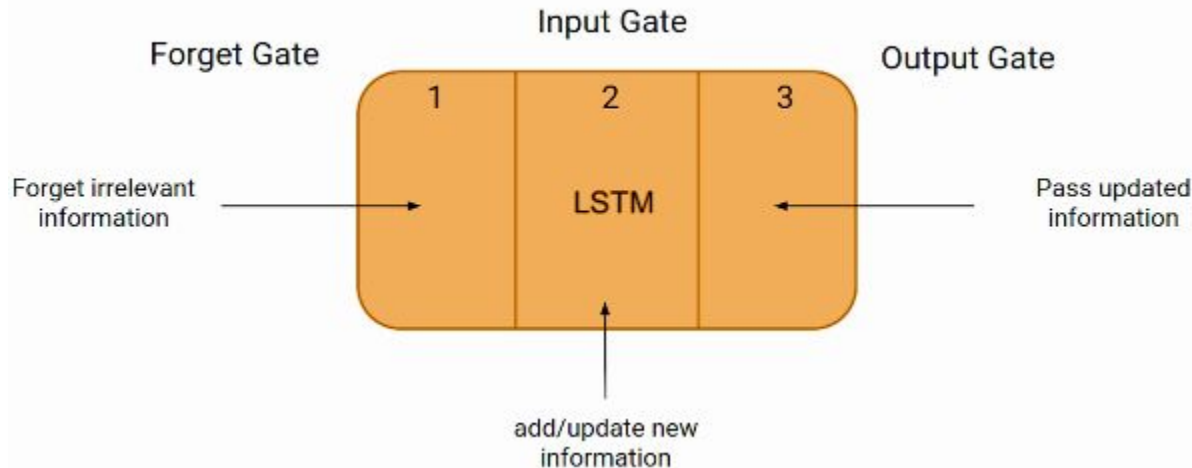
What is an RNN?

- Imagine a neural network that not only processes the current input but also takes into account information from previous inputs.
- **Sequential Memory**: RNNs have a hidden state that retains information about the sequence,
- **Challenges**:
 - Gradient flow problems (vanishing or exploding gradients)
 - Long term memory loss
 - Computation cost

$$\text{New weights} = \text{old weights} + \text{learning rate} * \text{gradients}$$

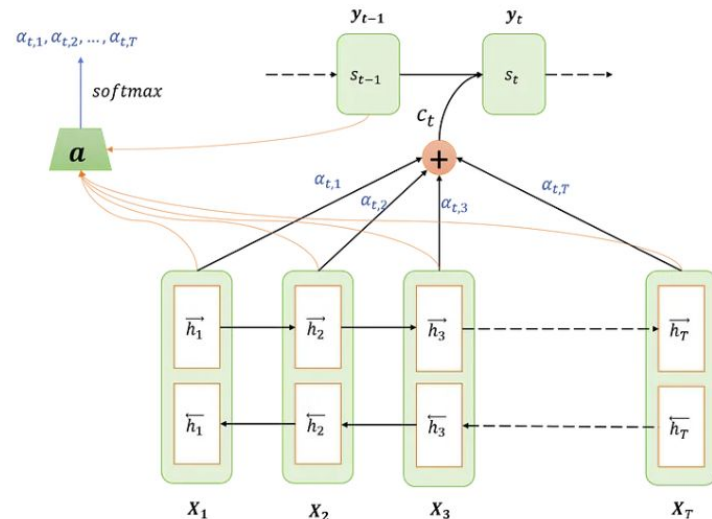
2 - Long Short Term Memories (LSTMs)

- Solves the vanishing gradient problem in RNNs
- Uses gating mechanisms
- Input gate, forget gate, output gate



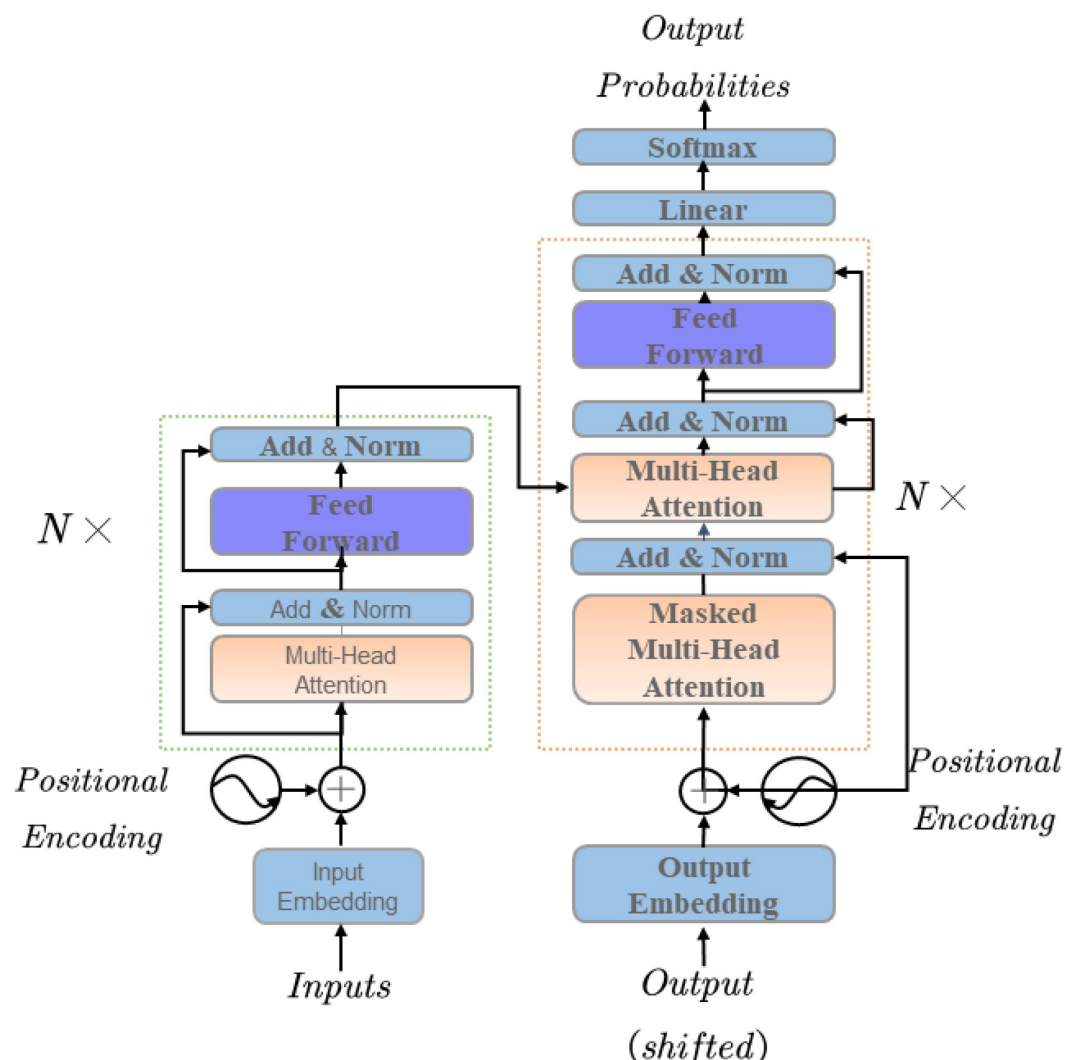
Attention Mechanism

- Too long sequences of data is still problematic
- This primarily happens because **memory is updated by means of a short-term change**, therefore **longer term relationships** are forgotten over time
- **Solution: use all intermediate states**, not just the last one for generating the output (use weighted states)
- **Long-term memory loss fixed!**



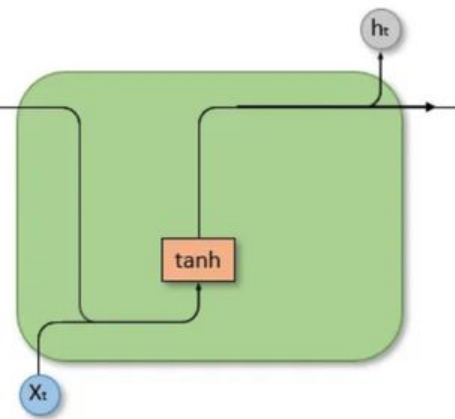
3 - Transformers

- But ...What remains problematic is: Sequential processing is computationally expensive (memory was improved using attention mechanism, but computation was not)
- **Solution:**keep the benefits of attention, but reduce computational costs by a new architecture called transformers
- It's an **encoder-decoder** model

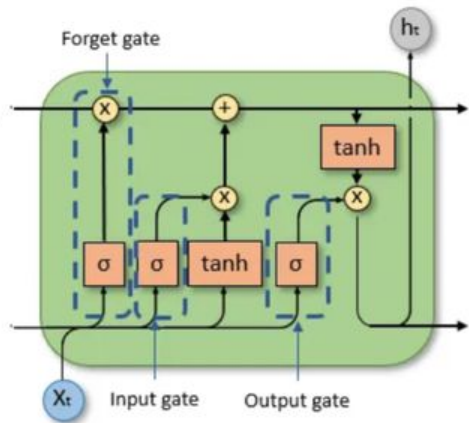


An overview of the NNs for temporally correlated data

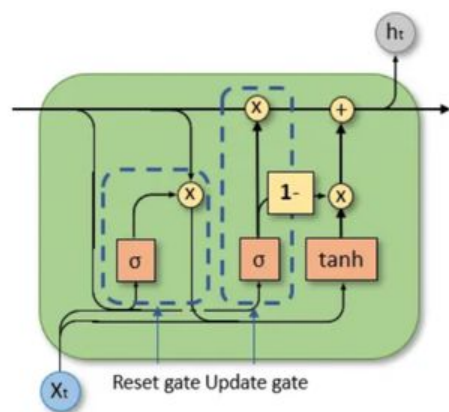
RNN



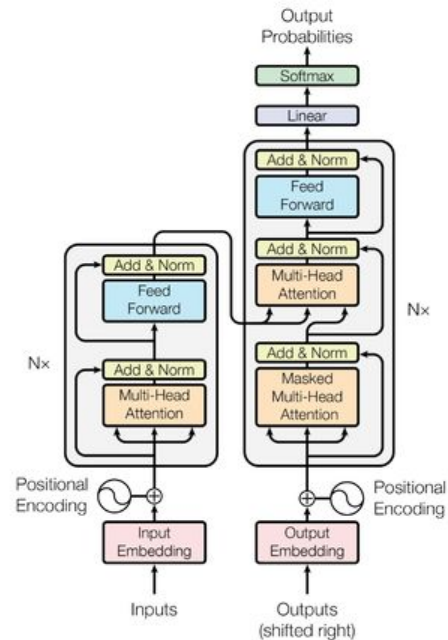
LSTM



GRU



Transformers

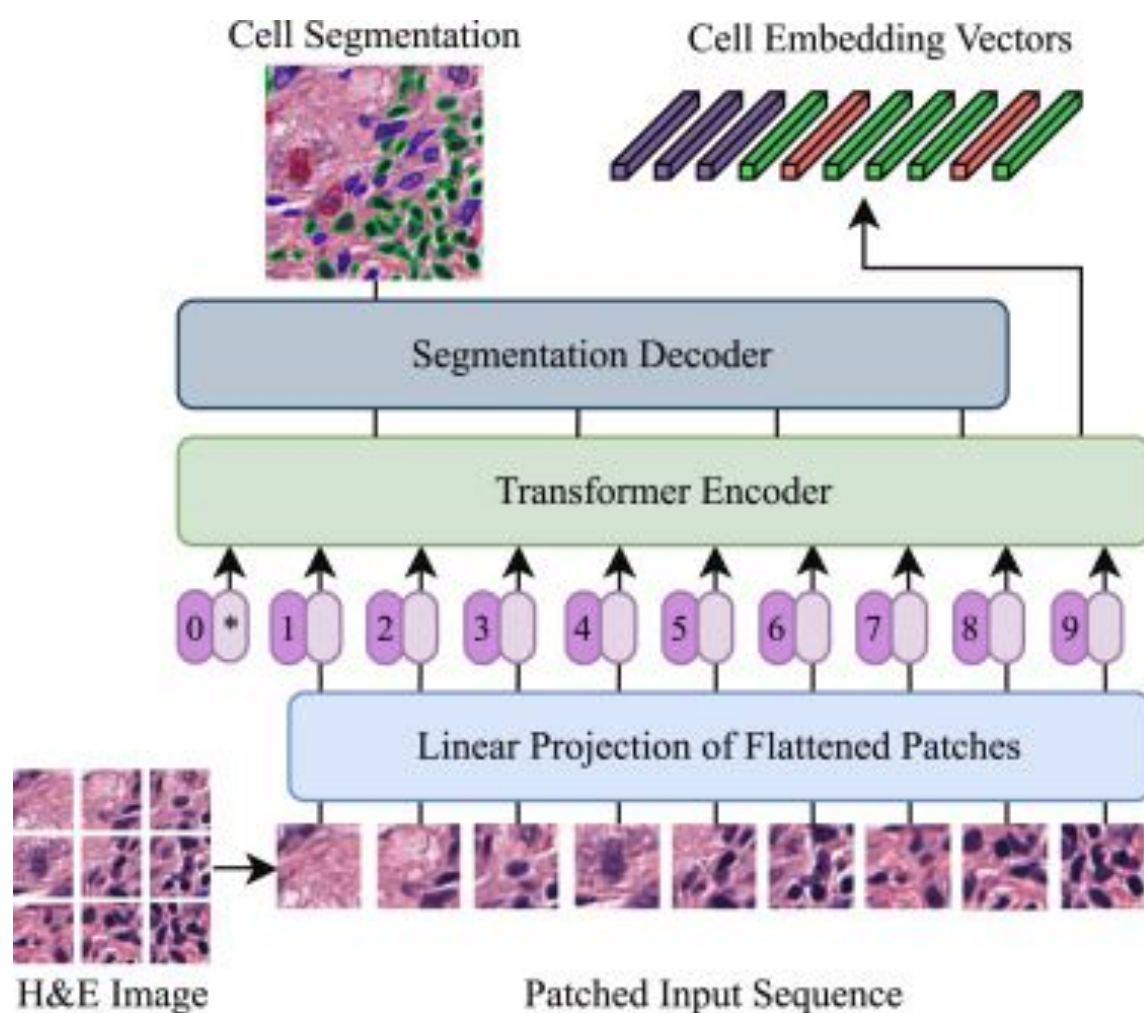


Vision Transformers (ViT)

Let's modify transformers to process images

Let's Brainstorm!

What are the applications of ViT in medicine?



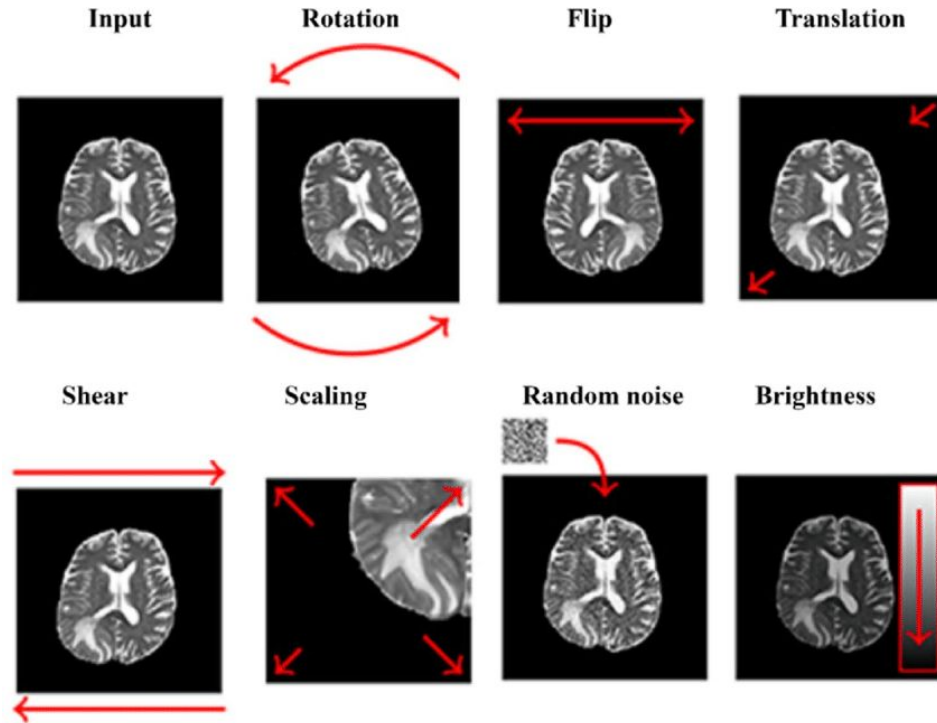
Neural Networks for Data Generation and Augmentation

Let's brainstorm!

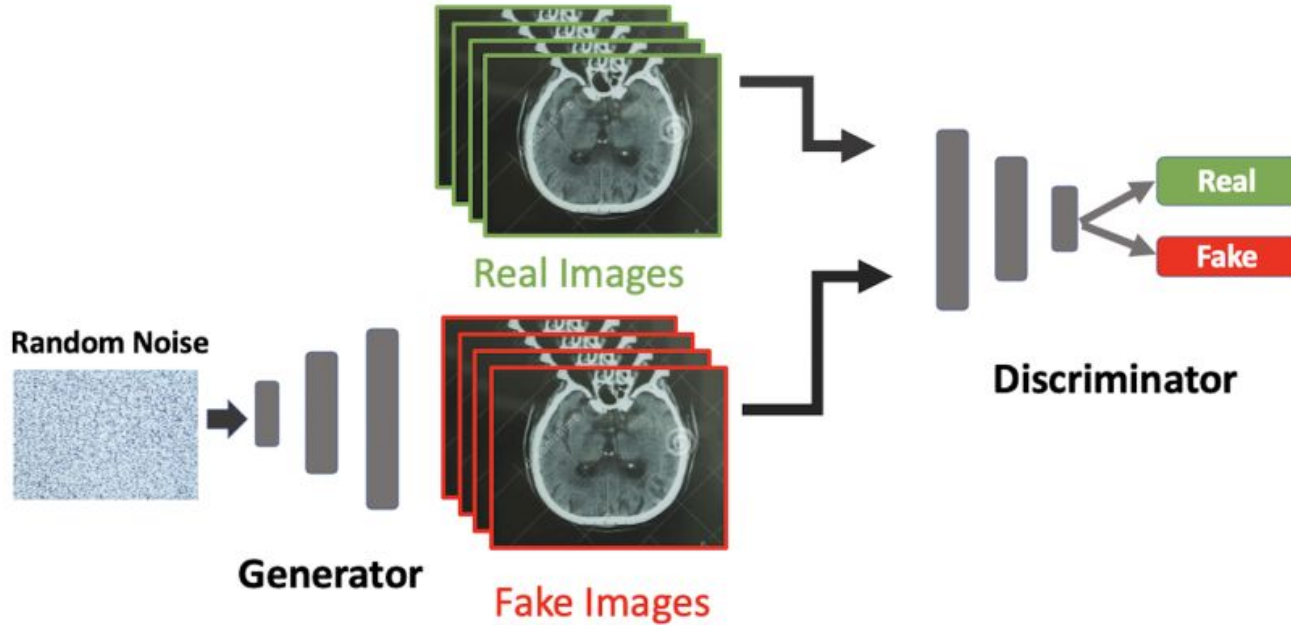
Why do we need data augmentation in healthcare applications?

Is it supervised, unsupervised, or reinforcement learning?

Regular data augmentation methods



1 - Generative Adversarial Networks (GANs)



2 - Diffusion Models

How Diffusion models work:

generative models that **“diffuse” training data with random noise**, then **learn to reverse the diffusion process** to output new images.

What they are used for:

- Image reconstruction
- Denoising
- Image generation
- Anomaly detection

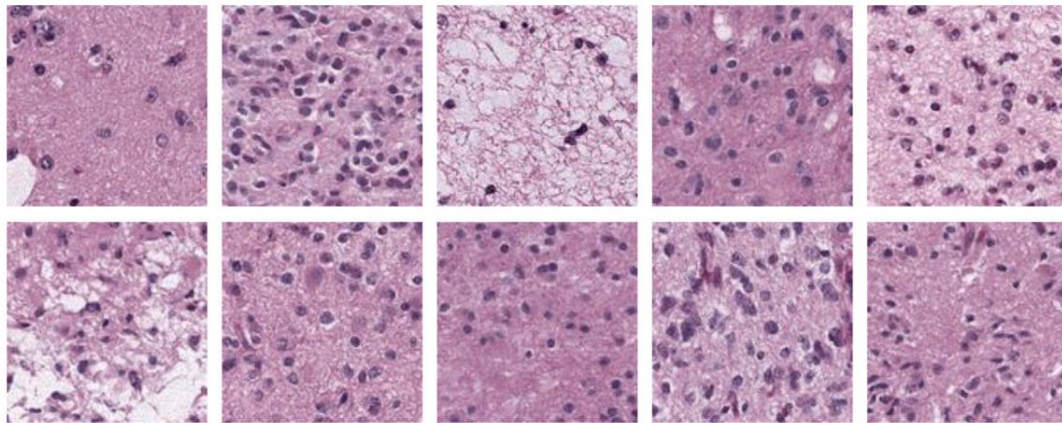
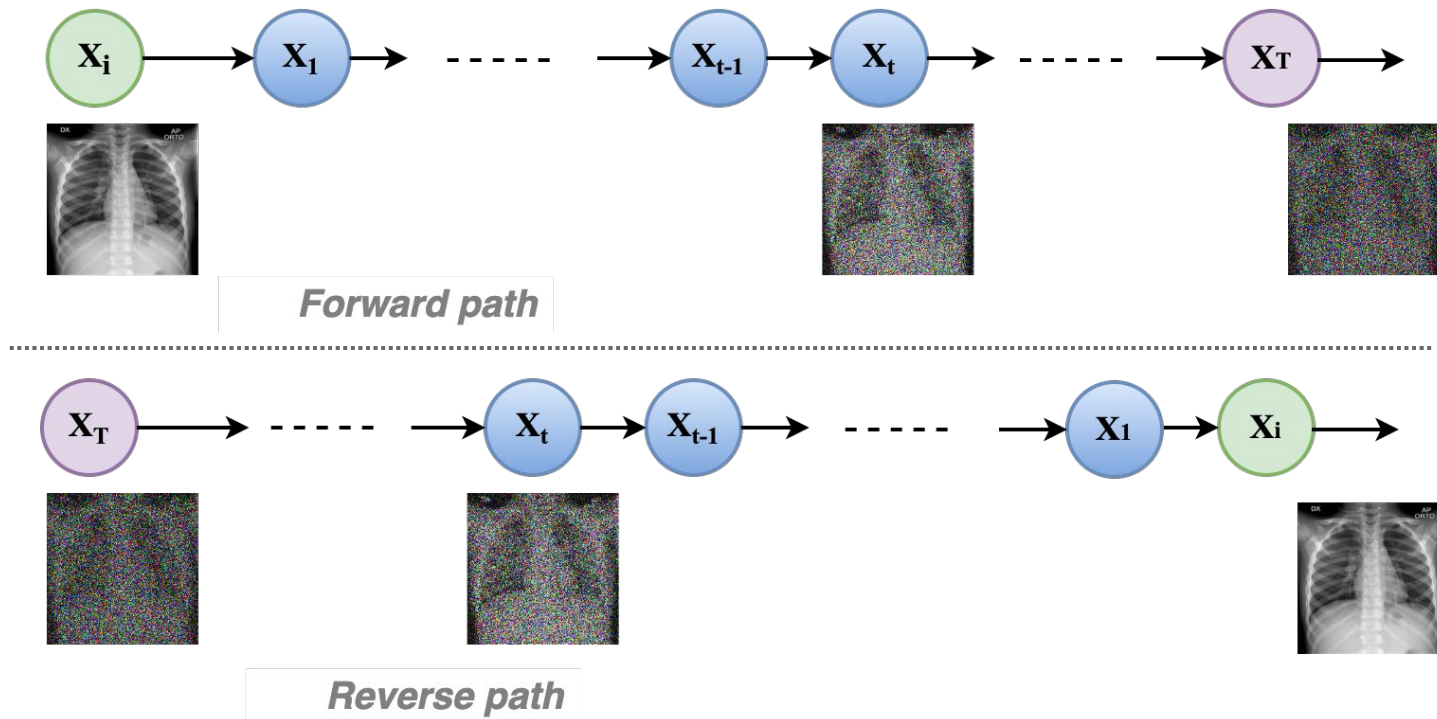


Fig. 4. Ten synthetic histopathology images generated by MFDPM (Moghadam et al., 2023).

2 - Diffusion Models



**Generative models can improve the
fairness and generalizability of medical
classifiers**

Neural Networks for Text and Language Data

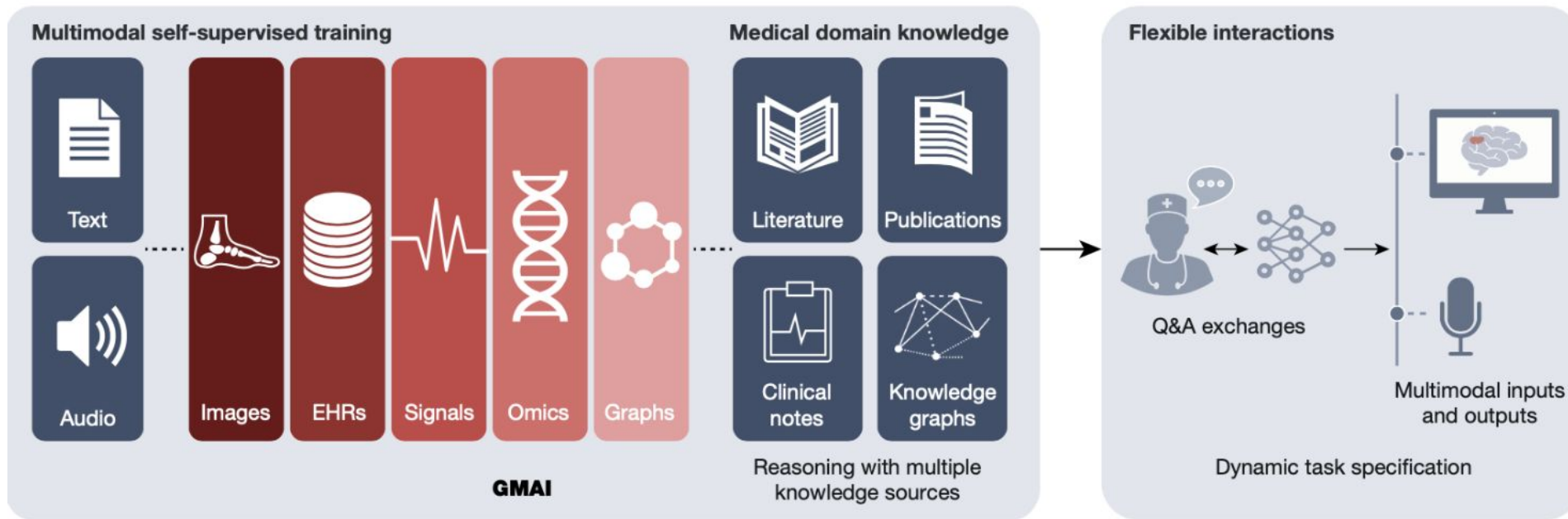
Have you used ChatGPT?

It's more than ChatGPT, it's the foundation models!

- LLMs are a type of foundation model, but they are not the only kind.
- Foundation models are:
 - Trained on **very large, diverse datasets**
 - Able to learn from **multiple data types, such as text, images, audio, and video (multimodal models)**
 - Trained using **self-supervised learning**, where the model learns without manual **labels** by:
 - Predicting **missing or masked parts of the input**
 - Discovering **patterns, relationships, and structure** in the data

Foundation Models: How They Work

- **Pre-training:** Learning **general patterns** from large, diverse datasets
Fine-tuning: Adapting the pre-trained model to a **specific task or domain** using a smaller, targeted dataset.
- **Alignment:** ensuring the model's outputs are: helpful, harmless, and **aligned with human intent and values**

a**b****Applications**

Chatbots for patients



Interactive note-taking



Augmented procedures

...



Grounded radiology reports



Text-to-protein generation



Bedside decision support

Foundation Models: Pros and Cons

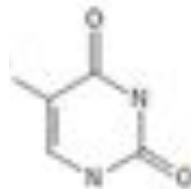
Pros:

- Adaptability to a wide range of tasks
- High accuracy on various tasks

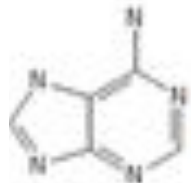
Cons:

- Bias and fairness
- Hallucinations
- High computational cost

Neural Networks for Graph-Structured Data



Drug A



Drug B

Graph-Structured Data in Medicine

- Sometimes, data is not **organized as tables or images**
- Instead, it has a **relational structure**, where **concepts** are **connected** to one another
 - **Concepts** are called **nodes**
 - **Connections** between concepts are called **edges**
- This forms a graph representation of the data
- To work with this structure, we need **specialized models designed for graphs**
- **Graph Neural Networks (GNNs)** are used when data is naturally represented as a graph

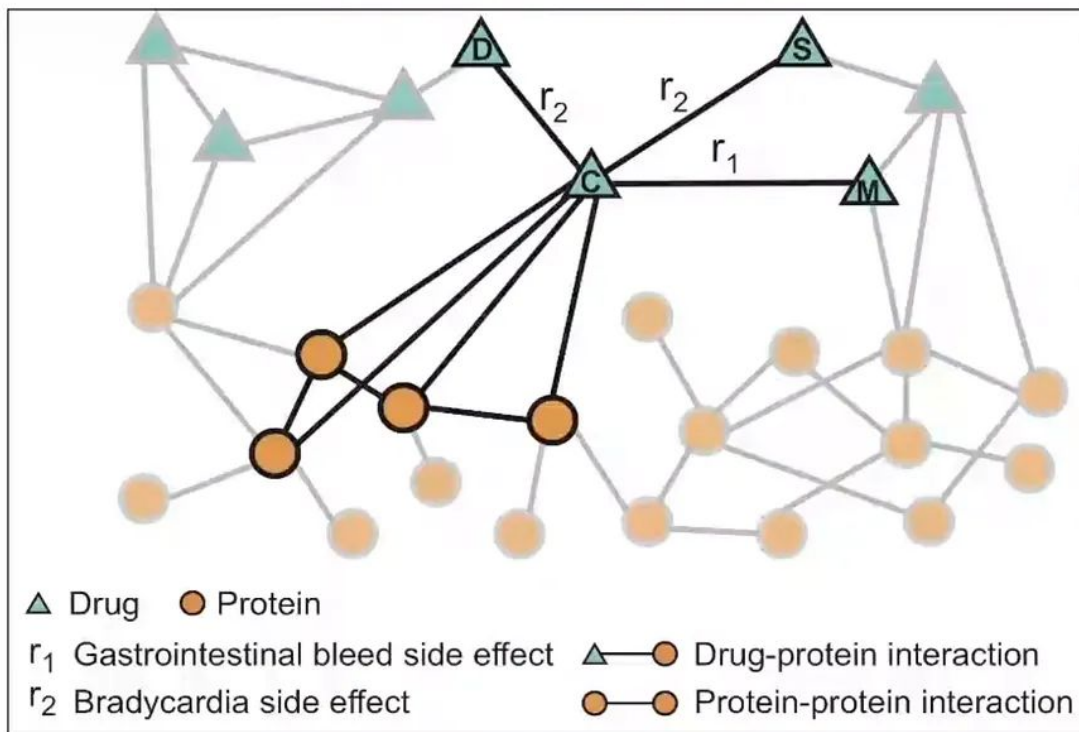
Graph-Structured Data in Medicine

Examples:

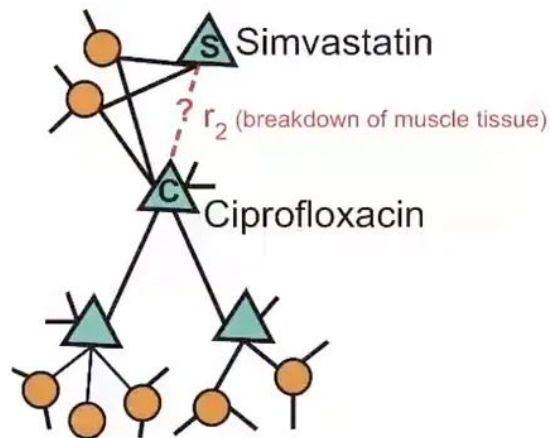
- Molecular structures of drugs
- Drug-drug interaction networks
- Hierarchical organization of cell systems
- Gene interaction networks

What do the nodes and edges represent in each of these examples?

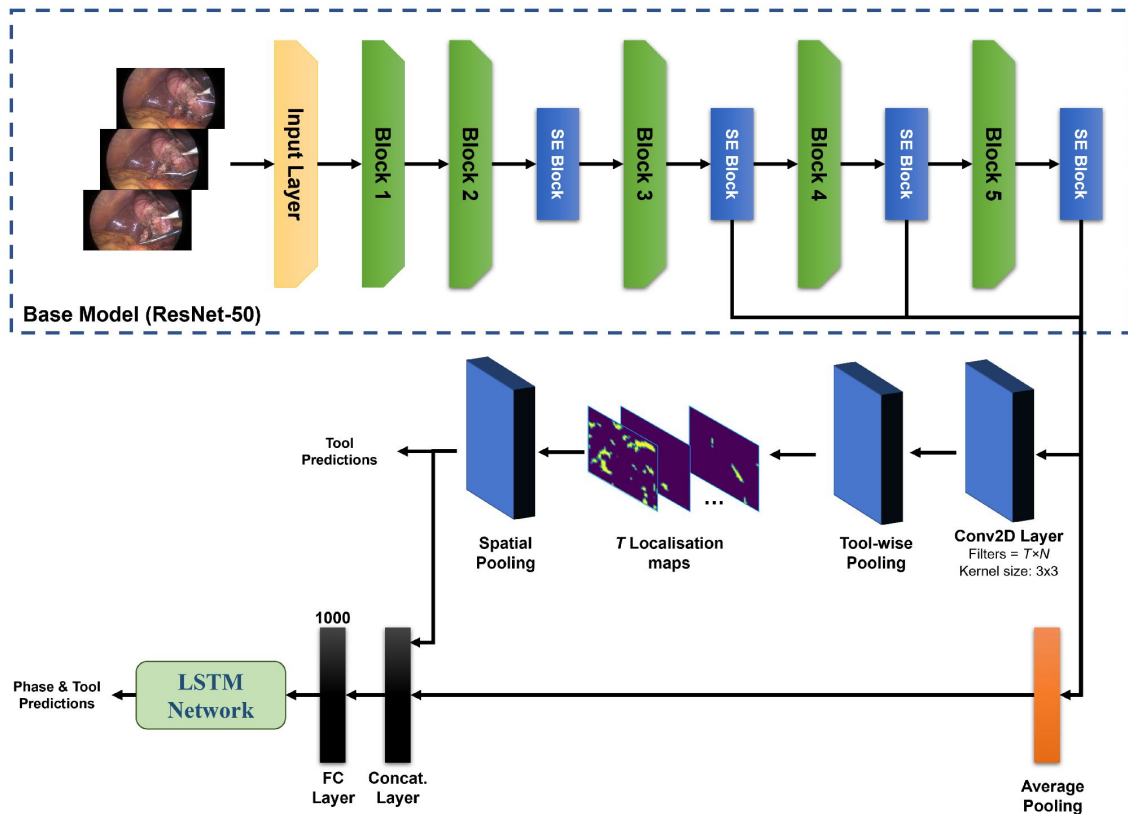
- **Nodes:** Drugs & Proteins
- **Edges:** Interactions



Query: How likely will Simvastatin and Ciprofloxacin, when taken together, break down muscle tissue?



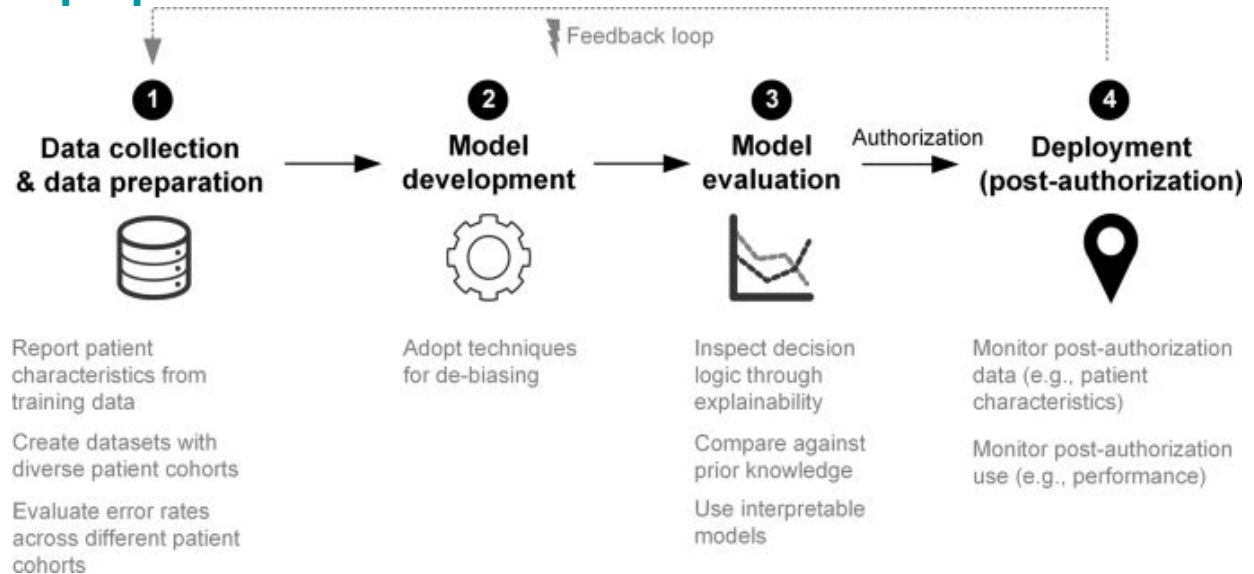
You can mix and match these components



Biases, Failures, and Considerations

Bias in ML-based systems for medical applications can occur across the different development steps:

1. data collection and data preparation
2. model development
3. model evaluation
4. Model deployment



Biases, Failures, and Considerations: **Data Collection**

- **Unrepresentative Data:** If the patient cohort used for training does not reflect the target population, the ML system may perform poorly or unfairly.
- **Example:** Training data may lack diversity in age, ethnicity, or socioeconomic status, leading to biased outcomes.

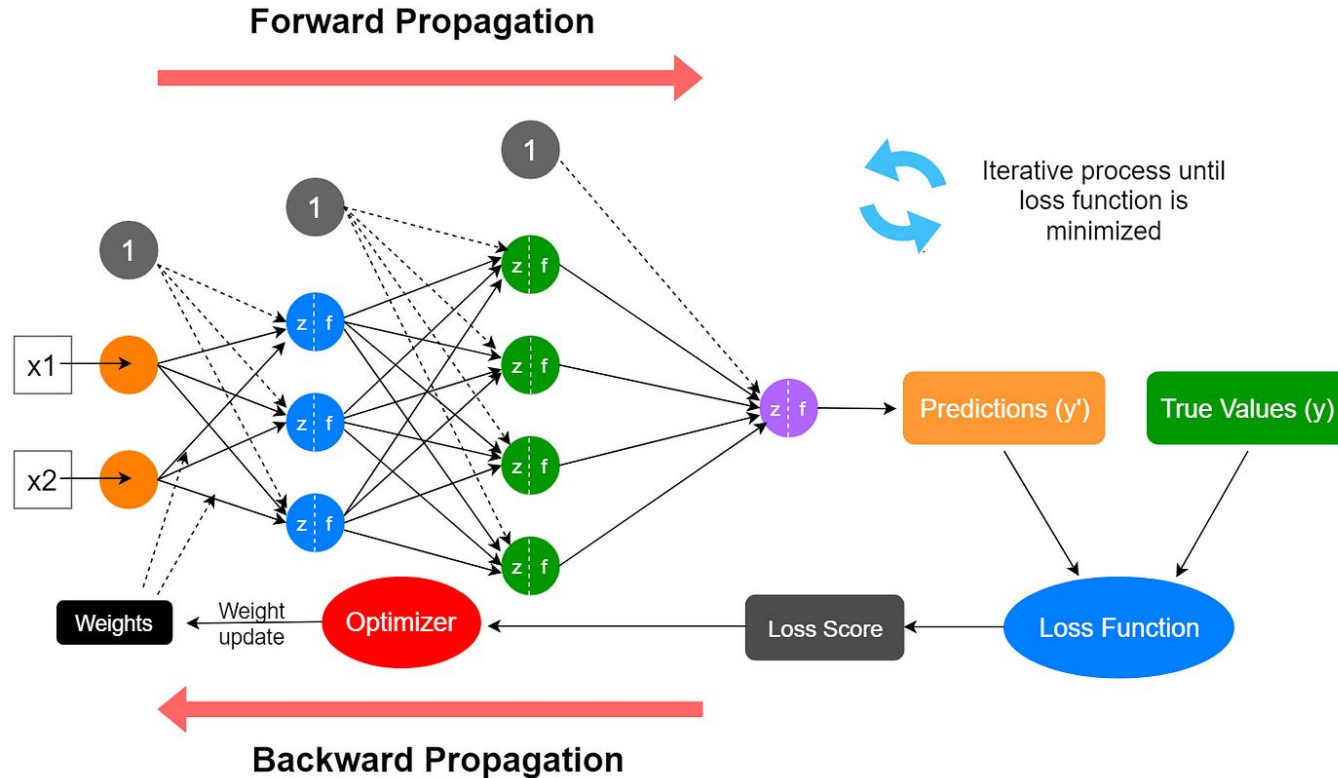
Biases, Failures, and Considerations: **Model Development**

- **De-biasing Techniques:** Mathematical methods can reduce bias during model training.
- **Oversampling:** Balances underrepresented groups by creating synthetic data points.

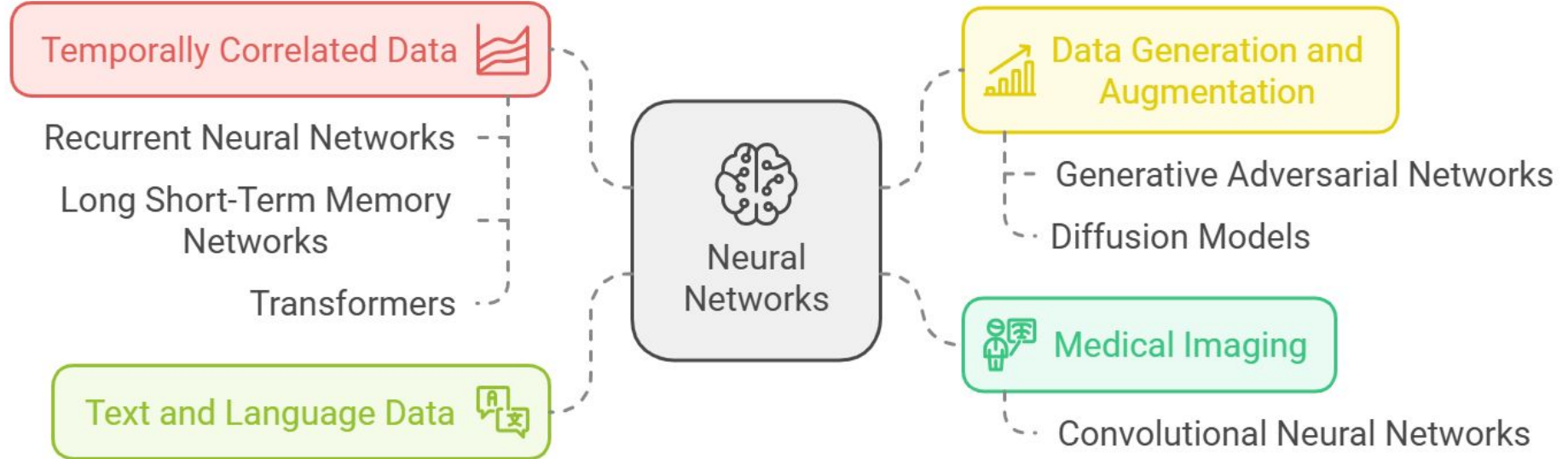
Biases, Failures, and Considerations: **Explainability**

- Clinicians need to trust and understand the AI system's predictions.
- Complex models (e.g., deep neural networks) often lack transparency.
- Use interpretable models or post-hoc techniques like SHAP or LIME to provide insights into decision-making.

Summary: Fundamentals



Summary: Applications



Summary

- Different types of **data** require different machine learning **models**
- **Medical domain knowledge** is essential for correctly **formulating the machine learning problem**
- When thoughtfully developed and deployed, these models can support:
 - **Individual clinicians**
 - **Public health planning**
 - **The general public**
- Through **careful evaluation** and by identifying **biases and risks**, we can ensure these models **benefit practitioners and patients safely**

Thank you!

What are foundation models?

<https://iml.dfki.de/foundation-models-for-medical-ai/>

<https://www.nature.com/articles/s41746-025-01924-4>

<https://hai.stanford.edu/news/shaky-foundations-foundation-models-healthcare>

<https://cloud.google.com/discover/what-are-foundation-models>

Added: graph neural networks

<https://snap.stanford.edu/decagon/>

<https://grlearning.github.io/slides/zitnik.pdf>

<https://www.nature.com/articles/s41598-024-83090-3>